

飞腾嵌入式 OpenAMP 用户使用手册

(V1.7.1)

Phytium 飞腾

飞腾信息技术有限公司

www.phytium.com.cn

2026 年 3 月 30 日

版权所有 © 飞腾信息技术有限公司 2022-2026。 保留一切权利。

未经本公司同意，任何单位、公司或个人不得擅自复制、翻译、摘抄本文档内容的部分或全部，不得以任何方式或途径进行传播和宣传。

商标声明

Phytium 和其他飞腾商标均为飞腾信息技术有限公司的商标。

本文档提及的其他所有商标或注册商标，由各自的所有人拥有。

特别提示

本文档仅作为使用指导，飞腾对本文档内容不做任何明示或暗示的声明或保证。本文档中的所有陈述、信息和建议不构成任何明示或暗示的担保。

由于产品版本升级或其他原因，本文档内容会不定期进行更新，如有变更，恕不另行通知。

最新技术资源

飞腾嵌入式软件开源社区 https://gitee.com/phytium_embedded

或者访问飞腾软件开发者平台 <https://service.phytium.com.cn/developer/>

目录

1 关于飞腾嵌入式 OpenAMP	3
1.1 官方网址	3
1.2 CPU 支持	3
1.3 应用场景	3
2 用户操作	4
2.1 Linux 侧操作	4
2.1.1 获取飞腾嵌入式 linux kernel	4
2.1.2 获取应用程序测试代码	4
2.1.3 配置设备树	4
2.1.4 编译设备树	7
2.1.5 U-Boot 启动方式	8
2.1.6 UEFI + DTS 启动方式	8
2.1.7 单个 rproc 功能测试	9
2.1.8 两个 rproc 功能测试	10
2.2 实时系统侧操作	11
2.2.1 应用场景之 type-A 操作	11
2.2.2 应用场景之 type-B 操作	13
2.2.3 应用场景之 type-C 操作	15
2.2.4 应用场景之 type-D 操作	15
2.2.5 应用场景之 type-E 操作	15
A 术语	15
B 更新记录	16

Phytium 飞腾

1 关于飞腾嵌入式 OpenAMP

在 OpenAMP 开源软件的基础上，在飞腾 CPU 上提供运行两个或三个系统（Linux+RTOS, RTOS+RTOS）的方案。

1.1 官方网址

OpenAMP 官方：

<https://www.openampproject.org/>

<https://github.com/OpenAMP/open-amp>

<https://github.com/OpenAMP/libmetal>

飞腾官方：

https://gitee.com/phytium_embedded/phytium-standalone-sdk

1.2 CPU 支持

E2000, D2000, FT-2000/4, D3000, D3000M

1.3 应用场景

type	master	slave	备注
A	Linux kernel	baremetal	open-amp、libmetal
B	Linux kernel	freertos	open-amp、libmetal
C	baremetal	barmetal	open-amp、libmetal
D	freertos	freertos	open-amp、libmetal
E	baremetal	baremetal	仅使用 libmetal 库

2 用户操作

2.1 Linux 侧操作

下面的步骤在 E2000Q、D2000、FT-2000/4、D3000、D3000M 上均适用，目前 5.10 和 6.6 普通内核上功能完整，不推荐在 4.19 内核和 RT 内核上进行测试。另外需要注意的是，目前提供的新版本支持两个 rproc 功能测试，后续操作中有注解哪些操作是在进行两个 rproc 功能测试时需要做的，而在单个 rproc 测试中并不需要，请仔细区分。

2.1.1 获取飞腾嵌入式 linux kernel

请用户到飞腾嵌入式官方下载或者通过飞腾嵌入式相关工作人员获取。

内核源码下载地址：

5.10: https://gitee.com/phytium_embedded/phytium-linux-kernel/tree/linux-5.10/

6.6: https://gitee.com/phytium_embedded/phytium-linux-kernel/tree/linux-6.6/

注意：请使用最新稳定版本内核，目前对应的 commit 点为：

d964a22b43c90f8e1607678e8d4b84489333a667 (5.10) ；

00389a4b203236b1edb038bf5762364ce55ab081 (tag: kernel-6.6_v3.3) 。

2.1.2 获取应用程序测试代码

飞腾嵌入式 OpenAMP 实现了 rpmsg-demo-xxx.c 源代码，供用户参考，源代码位置在

https://gitee.com/phytium_embedded/phytium-embedded-docs/blob/master/open-amp/rpmsg-demo-single.c

https://gitee.com/phytium_embedded/phytium-embedded-docs/blob/master/open-amp/rpmsg-demo-double.c

注意：原 rpmsg-demo.c 已不适合 “openamp-v1.0” tag 点之后编译的镜像测试！

2.1.3 配置设备树

配置设备树是为了预留内存以及使能 homo_rproc 节点。

E2000Q 设备树文件为：pe220x.dtsi、pe2204.dtsi、e2000q-demo-board.dts

D2000 设备树文件为：pd2008-generic-psci-soc.dtsi、pd2008-devboard-dsk.dts

FT-2000/4 设备树文件为：pd1904-generic-psci-soc.dtsi、pd1904-devboard-d4-dsk.dts

D3000 设备树文件为：pd2308.dtsi、pd2308-demo-a.dts

D3000M 设备树文件为：pd2408-generic-psci-soc.dtsi、pd2408-hanwei-onboard.dts

其中 D2000、FT-2000/4、D3000、D3000M 设备树文件未开源，如有需要请联系 linan1284@phytium.com.cn。

1) 设备树的内核配置文档：

Documentation/devicetree/bindings/remoteproc/homo-rproc.txt;

2) 保留内存设置：

E2000Q 在 pe220x.dtsi 中设置保留内存

D2000 在 pd2008-generic-psci-soc.dtsi 中设置保留内存

FT-2000/4 在 pd1904-generic-psci-soc.dtsi 中设置保留内存

D3000 在 pd2308-demo-a.dts 中设置保留内存

D3000M 在 pd2408-hanwei-onboard.dts 中设置保留内存

添加内容如下：（注意：请将下面的内容添加在设备树根节点）

（a）如果只测试一个 rproc 功能，dtsi 修改如下：

```
reserved-memory {
    #address-cells = <0x2>;
    #size-cells = <0x2>;
    ranges;
    rproc0: rproc0@b0100000 {
        no-map;
        reg = <0x0 0xb0100000 0x0 0x19900000>;
    };
};

homo_rproc: homo_rproc@0 {
    compatible = "homo,rproc";
    status = "disabled";
    homo_core0: homo_core0@b0100000 {
        compatible = "homo,rproc-core";
        remote-processor = <3>;
        inter-processor-interrupt = <9>;
        memory-region = <&rproc0>;
        firmware-name = "openamp_core0.elf";
    };
};
```

(b) 如果测试两个 rproc 功能，dtsi 修改如下：

```
reserved-memory {
    #address-cells = <0x2>;
    #size-cells = <0x2>;
    ranges;
    rproc0: rproc0@b0100000 {
        no-map;
        reg = <0x0 0xb0100000 0x0 0x19900000>;
    };
    rproc1: rproc1@d0100000 {
        no-map;
        reg = <0x0 0xd0100000 0x0 0x19900000>;
    };
};

homo_rproc: homo_rproc@0 {
    compatible = "homo,rproc";
    status = "disabled";
    homo_core0: homo_core0@b0100000 {
        compatible = "homo,rproc-core";
        remote-processor = <3>;
        inter-processor-interrupt = <9>;
        memory-region = <&rproc0>;
        firmware-name = "openamp_core0.elf";
    };
    homo_core1: homo_core1@d0100000 {
        compatible = "homo,rproc-core";
        remote-processor = <2>;
        inter-processor-interrupt = <10>;
        memory-region = <&rproc1>;
        firmware-name = "openamp_core1.elf";
    };
};
```

其中：“remote-processor”项指定 cpuid，“inter-processor-interrupt”项指定 IPI/SPI 中断号，“firmware-name”指定 openamp 镜像名称。例如，在上述 dtsi 改动中，rproc0 使用的是 9 号 IPI 中断，openamp 镜像名称为“openamp_core0.elf”，而 rproc1 使用的是 10 号 IPI 中断，openamp 镜像名称为“openamp_core1.elf”，请仔细区分。

注意，上述设备树节点需要 enable。

E2000Q 在 e2000q-demo-board.dts（5.10 内核）、pe2204-demo-ddr4.dts（6.6 内核）

中添加节点;

D2000 在 pd2008-devboard-dsk.dts 中添加节点;

FT-2000/4 在 pd1904-devboard-d4-dsk.dts 中添加节点;

D3000 在 pd2308-demo-a.dts 中添加节点;

D3000M 在 pd2408-hanwei-onboard.dts 中添加节点;

添加内容如下:

注意: 请将下面的内容直接添加在设备树文件内容末尾即可。

```
&homo_rproc {  
    status = "okay";  
};
```

2.1.4 编译设备树

1) E2000Q:

E2000Q 设备树文件在内核路径 arch/arm64/boot/dts/phytium 下, 编译内核之后即可得到 e2000q-demo-board.dtb 文件 (5.10 内核)、pe2204-demo-ddr4.dtb 文件 (6.6 内核)。

2) D2000:

将 pd2008-generic-psci-soc.dtsi 和 pd2008-devboard-dsk.dts 这两个设备树文件放到内核目录 arch/arm64/boot/dts/phytium 下, 编译内核之后即可得到 pd2008-devboard-dsk.dtb 文件。

3) FT-2000/4:

将 pd1904-generic-psci-soc.dtsi 和 pd1904-devboard-d4-dsk.dts 这两个设备树文件放到内核目录 arch/arm64/boot/dts/phytium 下, 编译内核之后即可得到 pd1904-devboard-d4-dsk.dtb 文件。

4) D3000:

将 pd2308.dtsi 和 pd2308-demo-a.dts 这两个设备树文件放到内核目录 arch/arm64/boot/dts/phytium 下, 编译内核之后即可得到 pd2308-demo-a.dtb 文件。

5) D3000M:

将 pd2408-generic-psci-soc.dtsi 和 pd2408-hanwei-onboard.dts 这两个设备树文件放到内核目录 arch/arm64/boot/dts/phytium 下, 编译内核之后即可得到 pd2408-hanwei-onboard.dtb 文件。

2.1.5 U-Boot 启动方式

以 FT-Demo_D2000_V0.1 开发板为例进行演示，使用 2.1.4 节编译出来的设备树文件搭建 D2000 开发板启动环境。E2000Q 和 FT-2000/4 更换对应的设备树文件即可。

- 1) 挂载已有磁盘的引导分区，更换设备树文件：

```
# mount /dev/sda1 /mnt
# cp pd2008-devboard-dsk.dtb /mnt
# sync
# umount /mnt
```

- 2) 重启系统：

```
# init 6
```

- 3) 在 U-Boot 命令行中输入内核命令行参数：

```
# setenv bootargs console=ttyAMA1,115200 earlycon=pl011,0x28001000 root=/dev/sda2
rootdelay=5 rw
```

- 4) 在 U-Boot 命令行中输入如下命令：

```
# ext4load scsi 0:1 0x90100000 pd2008-devboard-dsk.dtb
# ext4load scsi 0:1 0x90200000 Image
# booti 0x90200000 - 0x90100000
```

注意：

- 1、如上所述的两个文件 pd2008-devboard-dsk.dtb、Image 必须放在分区/dev/sda1 (不是分区 sda2)。

- 2、上述参数"earlycon=pl011,0x28001000",其中 0x28001000 为芯片调试串口的物理地址。启动时需要根据芯片类型进行实时修改，具体如下：

E2000Q: 0x2800d000

D2000: 0x28001000

FT-2000/4: 0x28001000

D3000: 0x28009000

D3000M: 0x18002000

2.1.6 UEFI + DTS 启动方式

以 FT-Demo_D2000_V0.1 开发板为例进行演示，使用 2.1.4 节编译出来的设备树文件搭建 D2000 开发板启动环境。E2000Q 和 FT-2000/4 更换对应的设备树文件即可。

- 1) 修改 UEFI 引导文件 EFI/BOOT/grub.cfg，添加如下内容：

```
menuentry 'boot'{
    devicetree /pd2008-devboard-dsk.dtb
```

```
}
```

- 2) 将 pd2008-devboard-dsk.dtb、Image 放在分区/dev/sda1，重启系统：

```
# init 6
```

注意：

- 1、如上所述的两个文件 pd2008-devboard-dsk.dtb、Image 必须与 EFI 目录放在分区/dev/sda1 (EFI 分区)。
- 2、芯片调试串口的物理地址参数调整与 U-Boot 启动相同。

2.1.7 单个 rproc 功能功能测试

- 1) 编译镜像

生成 type-A 类型 open-amp 二进制镜像的步骤请参考 2.2.1 节

生成 type-B 类型 open-amp 二进制镜像的步骤请参考 2.2.2 节

镜像文件名：openamp_core0.elf

注意：镜像文件名需与设备树节点“firmware-name”匹配！

- 2) 拷贝镜像

```
# cp openamp_core0.elf /lib/firmware/
```

- 3) 启动镜像

注意：如果镜像已经启动，则先按照 5) 停止镜像运行！

```
# echo start > /sys/class/remoteproc/remoteproc0/state
# echo rpmsg_chrdev >
/sys/bus/rpmsg/devices/virtio0.rpmsg-openamp-demo-channel.-1.0/driver_override
# modprobe rpmsg_char
```

- 4) 执行测试程序

```
# gcc -o rpmsg-demo-single rpmsg-demo-single.c
# ./rpmsg-demo-single
```

注意：原 rpmsg-demo.c 已不适合“openamp-v1.0” tag 点之后编译的镜像测试！

- 5) 停止镜像运行

```
# echo stop > /sys/class/remoteproc/remoteproc0/state
# rmmod rpmsg_char
# rmmod rpmsg_ctrl // 6.6 内核才需要执行此步骤，5.10 内核无需执行
```

- 6) 运行结果截图：

```
root@3000:~/linux-kernel# echo start > /sys/class/remoteproc/remoteproc0/state
root@3000:~/linux-kernel# echo rpmsg_chrdev > /sys/bus/rpmsg/devices/virtio0.rpmsg-openamp-demo-channel--1.0/driver_override
root@3000:~/linux-kernel# modprobe rpmsg_char
root@3000:~/linux-kernel# ./rpmsg-demo-single
argc: 1
Argument 0: ./rpmsg-demo-single
received message: Hello World! No:1KP
received message: Hello World! No:2
received message: Hello World! No:3
received message: Hello World! No:4
received message: Hello World! No:5
received message: Hello World! No:6
received message: Hello World! No:7
received message: Hello World! No:8
received message: Hello World! No:9
received message: Hello World! No:10
received message: Hello World! No:11
```

2.1.8 两个 rproc 功能测试

1) 编译镜像

生成 type-A 类型 open-amp 二进制镜像的步骤请参考 2.2.1 节

生成 type-B 类型 open-amp 二进制镜像的步骤请参考 2.2.2 节

镜像文件名: **openamp_core0.elf** 和 **openamp_core1.elf**

注意: 镜像文件名需与设备树节点 “firmware-name” 匹配!

2) 拷贝镜像

```
# cp openamp_core0.elf /lib/firmware/
# cp openamp_core1.elf /lib/firmware/
```

3) 启动镜像

注意: 如果镜像已经启动, 则先按照 5) 停止镜像运行!

```
# echo start > /sys/class/remoteproc/remoteproc0/state
# echo start > /sys/class/remoteproc/remoteproc1/state
3)# echo rpmsg_chrdev >
/sys/bus/rpmsg/devices/virtio0.rpmsg-openamp-demo-channel--1.0/driver_override
4)# echo rpmsg_chrdev >
/sys/bus/rpmsg/devices/virtio1.rpmsg-openamp-demo-channel--1.0/driver_override
# modprobe rpmsg_char
```

4) 执行测试程序

```
# gcc -o rpmsg-demo-double rpmsg-demo-double.c
# ./rpmsg-demo-double
```

注意: 原 rpmsg-demo.c 已不适合 “openamp-v1.0” tag 点之后编译的镜像测试!

5) 停止镜像运行

```
# echo stop > /sys/class/remoteproc/remoteproc0/state
# echo stop > /sys/class/remoteproc/remoteproc1/state
# rmmod rpmsg_char
# rmmod rpmsg_ctrl // 6.6 内核才需要执行此步骤, 5.10 内核无需执行
```

6) 测试结果截图

```
root@phytium-Ubuntu:~/OpenAMP# echo start > /sys/class/remoteproc/remoteproc0/state
root@phytium-Ubuntu:~/OpenAMP# echo start > /sys/class/remoteproc/remoteproc1/state
root@phytium-Ubuntu:~/OpenAMP# echo rpmsg_chrdev > /sys/bus/rpmsg/devices/virtio0.rpmsg-openamp-demo-channel--1.0/driver_override
root@phytium-Ubuntu:~/OpenAMP# echo rpmsg_chrdev > /sys/bus/rpmsg/devices/virtio1.rpmsg-openamp-demo-channel--1.0/driver_override
root@phytium-Ubuntu:~/OpenAMP# modprobe rpmsg_char
root@phytium-Ubuntu:~/OpenAMP# ./rpmsg-demo-double
argc: 1
Argument 0: ./rpmsg-demo-double
received message: Hello World! No:1
rpmsg1 here received message: Hello World! No:101
received message: Hello World! No:2
rpmsg1 here received message: Hello World! No:102
received message: Hello World! No:3
rpmsg1 here received message: Hello World! No:103
received message: Hello World! No:4
rpmsg1 here received message: Hello World! No:104
received message: Hello World! No:5
rpmsg1 here received message: Hello World! No:105
received message: Hello World! No:6
```

2.2 实时系统侧操作

2.2.1 应用场景之 type-A 操作

提交与支持记录:

更新日期	链接地址	支持情况
2025/12/30	https://gitee.com/phytium_embedded/phytium-standalone-sdk/tree/openamp-v1.1	支持

- 1) 解压源码包(注意分支和提交号, *.zip 格式使用 unzip 命令):

```
# tar -xvf phytium-standalone-sdk-<TagVersion>.tar
```

- 2) phytium-standalone-sdk-<TagVersion> 在根目录下依据 README.md 中介绍, 安装 sdk 环境 (**环境很重要**), 推荐在本地主机上安装 sdk 开发环境, 然后将编译好的 *.elf 文件拷贝到开发板上。

- 3) 进入到具体目录位置进行编译:

```
# cd example/system/amp/openamp_for_linux
```

- 4) 加载默认的配置信息(不同芯片配置可参考本目录下的 README.md 文档进行配置), 此处以 E2000Q aarch64 为例:

```
# make config_pe2204_demo_aarch64
```

注意: 如果想要测试两个 rproc 功能, 那么需要修改配置文件, 然后编译用于第二个 rproc 的 elf 镜像文件, 以 E2000Q aarch64 为例, 需要修改以下三个文件:

- (a) configs/pe2204_aarch64_demo_openamp_for_linux.config

```
CONFIG_DEBUG_FULLOPT=y
CONFIG_DEBUG_SYMBOLS=y
CONFIG_DEFAULT_LINKER_SCRIPT=y
-CONFIG_IMAGE_LOAD_ADDRESS=0xb0100000
+CONFIG_IMAGE_LOAD_ADDRESS=0xd0100000
CONFIG_STACK_SIZE=0x1000
```

(b) common/libmetal_configs.h

```
--- a/example/system/amp/openamp_for_linux/common/libmetal_configs.h
+++ b/example/system/amp/openamp_for_linux/common/libmetal_configs.h
@@ -34,12 +34,12 @@ extern "C" {
    #include <metal/irq.h>

    #define KICK_BUS_NAME          "generic"
    -#define KICK_SGI_NUM_9          9
    +#define KICK_SGI_NUM          10
    // #define KICK_SGI_NUM_10      10

    /* device name for slave 00 */
    #define SLAVE_00_KICK_DEV_NAME "slave_00_kick"
    -#define SLAVE_00_SGI          KICK_SGI_NUM_9
    +#define SLAVE_00_SGI          KICK_SGI_NUM
    // #define SLAVE_01_KICK_DEV_NAME "slave_01_kick"
    // #define SLAVE_01_SGI          KICK_SGI_NUM_10
```

(c) common/memory_layout.h

```
--- a/example/system/amp/openamp_for_linux/common/memory_layout.h
+++ b/example/system/amp/openamp_for_linux/common/memory_layout.h
@@ -32,8 +32,8 @@ extern "C" {

    /*slave core0*/
    /* 与linux共享的内存 */
    -#define SLAVE00_SOURCE_TABLE_ADDR 0xc0000000 /*与linux协商好的地址*/
    -#define SLAVE00_KICK_IO_ADDR      0xc0224000
    +#define SLAVE00_SOURCE_TABLE_ADDR 0xe0000000 /*与linux协商好的地址*/
    +#define SLAVE00_KICK_IO_ADDR      0xe0224000
```

然后再执行:

```
# make config_pe2204_demo_aarch64
```

5) 编译 open-amp:

```
# make image
```

6) 按照所使用的芯片和开发板型号, 搭建并且启动 Linux 侧的环境, 详见章节 2.1 内容;

7) 两种不同编译平台, 推荐使用 b 方案, 在本地主机上编译后再将 openamp 拷贝到开发板上:

a) 开发板上编译裸机出的镜像: 生成的 elf 文件(默认放置/mnt/d/tftboot/目录下, 如果没有此目录, 可将生成的 pe2204_aarch64_demo_openamp_core0.elf 文件重命名为 openamp_core0.elf 文件), 并将生成的 'openamp_core0.elf' 拷贝至 /lib/firmware:

```
# cp openamp_core0.elf /lib/firmware/
```

如果是用于第二个 rproc 的 elf 镜像文件(双核 amp 方案, 需要 Linux 环境提前配置), 可将修改配置后重新生成的 pe2204_aarch64_demo_openamp_core0.elf 文件重命名为 openamp_core1.elf 文件, 并将生成的 'openamp_core1.elf' 拷贝至 /lib/firmware:

```
# cp openamp_core1.elf /lib/firmware/
```

b) 采取在本地主机编译镜像（推荐）：Windows 的 wsl 环境或者 Ubuntu 环境编译(详情见 2) 的 SDK 环境安装), 可以采用 scp 或者其他方式去拷贝到开发板上, 具体操作(举例):

需要先设置开发板的 ip 地址(此处设置 192.168.4.51, 如下图, 保持与本地主机 192.168.4.50 同一网段, 您可根据您本地主机地址进行 IP 地址的替换)。开发板端输入:

```
# ifconfig eth0 192.168.4.51
```

```
root@E2000-Ubuntu:~# ifconfig eth0 192.168.4.51
root@E2000-Ubuntu:~# ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.4.51 netmask 255.255.255.0 broadcast 192.168.4.255
    inet6 fe80::211:22ff:fe33:4455 prefixlen 64 scopeid 0x20<link>
    ether 00:11:22:33:44:55 txqueuelen 1000 (Ethernet)
    RX packets 160 bytes 10562 (10.5 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 72 bytes 8760 (8.7 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
    device interrupt 92 base 0xc000

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 288 bytes 21564 (21.5 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 288 bytes 21564 (21.5 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

root@E2000-Ubuntu:~#
```

在本地主机上将生成的 openamp_core0.elf 通过网络或者其他方式拷贝到开发板的 /lib/firmware 目录下, 此处本地主机使用 scp 方式为例(注意使用您生成的 elf 文件路径):

```
# scp /mnt/d/tftboot/openamp_core0.elf root@192.168.4.51:/lib/firmware/
```

输入 root 用户的 password: 完成拷贝。

2.2.2 应用场景之 type-B 操作

提交与支持记录:

更新日期	链接地址	支持情况
2025/12/30	https://gitee.com/phytium_embedded/phytium-free-rtos-sdk/tree/ophenamp-v1.1	支持

1) 解压源码包(注意分支和提交号, *.zip 格式使用 unzip 命令):

```
# tar -xvf phytium-free-rtos-sdk-<TagVersion>.tar
```

2) phytium-free-rtos-sdk-<TagVersion> 在根目录下依据 README.md 中介绍, 安装 sdk 环境 (环境很重要, 需要联网进行拉取裸机代码操作, 如果无网络, 则建议查阅 install.py 文件, 在有网络的地方下载 install.py 中变量 standalone_sdk_v 对应提交号的裸机代码解压后, 拷贝到 phytium-free-rtos-sdk 根目录并将文件夹命名为 standalone), 推荐在本地主机上安装 sdk 开发

环境，然后将编译好的*.elf 文件拷贝到开发板上。

3) 进入到具体目录位置进行编译：

```
# cd example/system/amp/openamp_for_linux
```

4) 加载默认的配置信息(不同芯片配置可参考本目录下的 README.md 文档进行配置)，此处以 E2000Q aarch64 为例：

```
# make config_pe2204_demo_aarch64
```

5) 编译 open-amp：

```
# make image
```

6) 按照所使用的芯片和开发板型号，搭建并且启动 Linux 侧的环境，章节 2.1 内容；

7) 两种不同编译平台，推荐使用 b 方案，在本地主机上编译后再拷贝到开发板上

a) 在开发板上编译裸机出的镜像：生成 elf 文件(默认放置/mnt/d/tftboot/目录下，如果没有此目录，可将生成的 pe2204_aarch64_demo_openamp_for_linux.elf 文件重命名为 openamp_core0.elf 文件)，并将生成的 'openamp_core0.elf' 拷贝至 Linux 开发板的 /lib/firmware：

```
# cp openamp_core0.elf /lib/firmware/
```

b) 采取在本地主机（推荐）：Windows 的 wsl 环境或者 Ubuntu 环境编译(详情见 2 的 SDK 环境安装)，可以采用 scp 或者其他方式去拷贝到开发板上，具体操作(举例)：

需要先设置开发板的 ip 地址(此处设置 192.168.4.51，如下图，保持与本地主机 192.168.4.50 同一网络段，您可根据您本地主机地址进行 IP 地址的替换)。开发板端输入：

```
ifconfig eth0 192.168.4.51
```

```
root@E2000-Ubuntu:~# ifconfig eth0 192.168.4.51
root@E2000-Ubuntu:~# ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
    inet 192.168.4.51  netmask 255.255.255.0  broadcast 192.168.4.255
    inet6 fe80::211:22ff:fe33:4455  prefixlen 64  scopeid 0x20<link>
    ether 00:11:22:33:44:55  txqueuelen 1000  (Ethernet)
    RX packets 160  bytes 10562 (10.5 KB)
    RX errors 0  dropped 0  overruns 0  frame 0
    TX packets 72  bytes 8760 (8.7 KB)
    TX errors 0  dropped 0  overruns 0  carrier 0  collisions 0
    device interrupt 92  base 0xc000

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
    inet 127.0.0.1  netmask 255.0.0.0
    inet6 ::1  prefixlen 128  scopeid 0x10<host>
    loop txqueuelen 1000  (Local Loopback)
    RX packets 288  bytes 21564 (21.5 KB)
    RX errors 0  dropped 0  overruns 0  frame 0
    TX packets 288  bytes 21564 (21.5 KB)
    TX errors 0  dropped 0  overruns 0  carrier 0  collisions 0

root@E2000-Ubuntu:~#
```

在本地主机上将生成的 openamp_core0.elf 通过网络或者其他方式拷贝到开发板的/lib/firmware

目录下，此处本地主机使用 scp 方式为例(注意使用您生成的 elf 文件路径)：

```
# scp /mnt/d/tftboot/openamp_core0.elf root@192.168.4.51:/lib/firmware/
```

输入 root 用户的 password: 完成拷贝。

2.2.3 应用场景之 type-C 操作

- 1) 解压源码包(注意分支和提交号, *.zip 格式使用 unzip 命令):

```
# tar -xvf phytium-standalone-sdk-<TagVersion>.tar
```
- 2) phytium-standalone-sdk-<TagVersion> 在根目录下依据 README.md 中介绍, 安装 sdk 环境 (**环境很重要**), 推荐在本地主机上安装 sdk 开发环境, 然后将编译好的*.elf 文件拷贝到开发板上。
- 3) 裸机之间参阅《多元异构系统部署 V1.1》, 看完 README.md 文档

2.2.4 应用场景之 type-D 操作

- 1) 解压源码包(注意分支和提交号, *.zip 格式使用 unzip 命令):

```
# tar -xvf phytium-free-rtos-sdk-<TagVersion>.tar
```
- 2) phytium-free-rtos-sdk-<TagVersion> 在根目录下依据 README.md 中介绍, 安装 sdk 环境 (**环境很重要, 需要联网进行拉取裸机代码操作, 如果无网络, 则建议查阅 install.py 文件, 在有网络的地方下载 install.py 中变量 standalone_sdk_v 对应提交号的裸机代码解压后, 拷贝到 phytium-free-rtos-sdk 根目录并将文件夹命名为 standalone**), 推荐在本地主机上安装 sdk 开发环境, 然后将编译好的*.elf 文件拷贝到开发板上。
- 3) 建议先学习 type-C 中的《多元异构系统部署 V1.1》, 查阅 README.md 文档

2.2.5 应用场景之 type-E 操作

- 1) 解压源码包(注意分支和提交号, *.zip 格式使用 unzip 命令):

```
# tar -xvf phytium-standalone-sdk-<TagVersion>.tar
```
- 2) phytium-standalone-sdk-<TagVersion> 在根目录下依据 README.md 中介绍, 安装 sdk 环境 (**环境很重要**), 推荐在本地主机上安装 sdk 开发环境, 然后将编译好的*.elf 文件拷贝到开发板上。
- 3) 进入到具体目录位置进行编译:

```
# cd example/system/amp/libmetal_test/apu_running
```

A 术语

对称多处理器系统 SMP(Symmetric Multi-Processing)

多应用于服务器/桌面应用领域;

系统中各个处理器主频率、cache 大小等相同。

非对称多处理器系统 AMP(Asymmetric Multi-Processing)

多应用工业嵌入式领域；

系统中的各个处理器通常主频率、cache 大小等不同。

B 更新记录

发布日期	说明
2022-02	v1.0 完成初稿
2022-03	v1.0 添加 FreeRTOS 和 baremetal 相关部分内容
2023-11	v1.1 更新 E2000 操作指导描述，以及完善测试步骤
2023-11	v1.2 更新 AMP 应用场景最新测试步骤
2023-12	v1.3 更新 E2000、D2000、FT-2000/4 操作指导描述，合并了重复部分。添加了 E2000、D2000、FT-2000/4 在 UEFI + DTS 启动方式下的 homo remoteproc 具体操作
2024-04	v1.4 添加了 Linux 内核源码版本说明、内核配置说明，细化了设备树节点添加方法以及 v1.3 版本错误修正
2025-03	V1.5 更新对 stop 功能以及两个 rproc 测试的支持；添加了 linux 动态加载裸机和 freertos 的说明，修改 type-C/D/E 的操作说明
2025-04	V1.6 测试文件更新，在原有使用到 rpmsg-demo.c 的章节中，修改其为 rpmsg-demo-single.c
2026-1	V1.7 增加 D3000 和 D3000M 相关内容；删除 type-C/D/E 的操作说明，改为链接指引；重新整理梳理文字和结构，提升可读性
2026-3	V1.7.1 修改错误章节的描述，更新了 pd2408 设备树文件名